

**Dokumentacja
Analiza Obrazów**

Rozpoznawanie odcisków palców



AGH

Wydział Fizyki i Informatyki Stosowanej

Kacper Mazurek

Radosław Iskra

Krystsina Mironenka

23.01.2026

Spis treści

1	Wstęp i cel projektu	3
2	Zespół projektowy	4
3	Środowisko pracy i architektura systemu	4
3.1	Stack technologiczny	4
3.2	Architektura systemu	4
4	Analiza etapów algorytmu	5
4.1	Warstwa przetwarzania wstępnego (Pre-processing)	6
4.1.1	Konwersja do skali szarości	7
4.1.2	Normalizacja	7
4.1.3	Segmentacja	8
4.1.4	Analiza polowa	9
4.1.5	Wzmocnienie	12
4.1.6	Wyniki przetwarzania wstępnego	14
4.2	Warstwa ekstrakcji cech	15
4.2.1	Binaryzacja obrazu	15
4.2.2	Szkieletyzacja obrazu	15
4.2.3	Wyniki binaryzacji i szkieletyzacji obrazu	16
4.3	Warstwa Post-processingu i Sygnatury	17
4.3.1	Detekcji minucji	17
4.3.2	Eliminacja fałszywych minucji	18
4.3.3	Sygnatura wektorowa	20
4.3.4	Wyniki Post-processingu	21
4.4	Porównywanie odcisków palców (Matching)	23
5	Testy programu	24
5.1	Ten sam palec (np. palec 101)	24
5.2	Różne palce (np. palce 101 vs 104)	24
5.3	Wnioski	25
6	Podsumowanie pracy	26
6.1	Wstępna obróbka	26
6.2	Ekstrakcja minucji i post-processing	26
6.3	Wnioski końcowe	26
7	Źródła i bibliografia	27

1 Wstęp i cel projektu

Biometria to dynamicznie rozwijająca się dziedzina, a rozpoznawanie odcisków palców stanowi jedną z najskuteczniejszych metod autentykacji dzięki unikalności punktów charakterystycznych, zwanych minucjami.

Głównym celem niniejszego projektu jest opracowanie i implementacja pełnego łańcucha algorytmicznego do rozpoznawania odcisków palców w środowisku MATLAB. Realizacja projektu skupia się na następujących aspektach:

- **Poprawa jakości obrazu (Pre-processing):** Opracowanie skutecznych metod wstępnego przetwarzania, takich jak normalizacja, segmentacja oraz filtracja Gabora, aby zredukować wariancję poziomów szarości i wzmocnić strukturę linii papilarnych.
- **Efektywna ekstrakcja cech:** Implementacja algorytmu wykrywania minucji opartego na metodzie Crossing Number (CN), umożliwiającego identyfikację zakończeń i rozwidleń linii.
- **Minimalizacja błędów:** Zastosowanie procedur eliminacji fałszywych minucji (np. powstałych na krawędziach obrazu lub w wyniku szumu), co pozwala na drastyczne obniżenie współczynnika błędów i zachowanie jedynie istotnych danych biometrycznych.
- **Optymalizacja procesu:** Stworzenie sygnatury wektorowej, która redukuje ilość przechowywanych informacji do niezbędnego minimum, co przyspiesza fazę porównywania i poprawia ogólną wydajność systemu.
- **Weryfikacja tożsamości:** Umożliwienie porównania dwóch sygnatur (zarejestrowanej i aktualnie pobranej) w celu potwierdzenia lub zaprzeczenia tożsamości użytkownika z uwzględnieniem akceptowalnego marginesu błędu.

2 Zespół projektowy

Poniższa tabela przedstawia podział obowiązków w zespole.

Tabela 1: Zakres obowiązków

Autor	Zakres pracy
Kacper Mazurek	Implementacja algorytmów wstępnego przetworzenia i analizy obrazu linii papilarnych.
Radosław Iskra	Implementacja algorytmów ekstrakcji minucji, post-processingu, wektora sygnatury, porównania odcisków
Krystyna Mironenka	Implementacja algorytmu usuwania fałszywych minucji, sporządzanie dokumentacji

3 Środowisko pracy i architektura systemu

Projekt został w całości zrealizowany i przetestowany w środowisku obliczeniowym MATLAB, które służy do walidacji algorytmów oraz wizualizacji wyników.

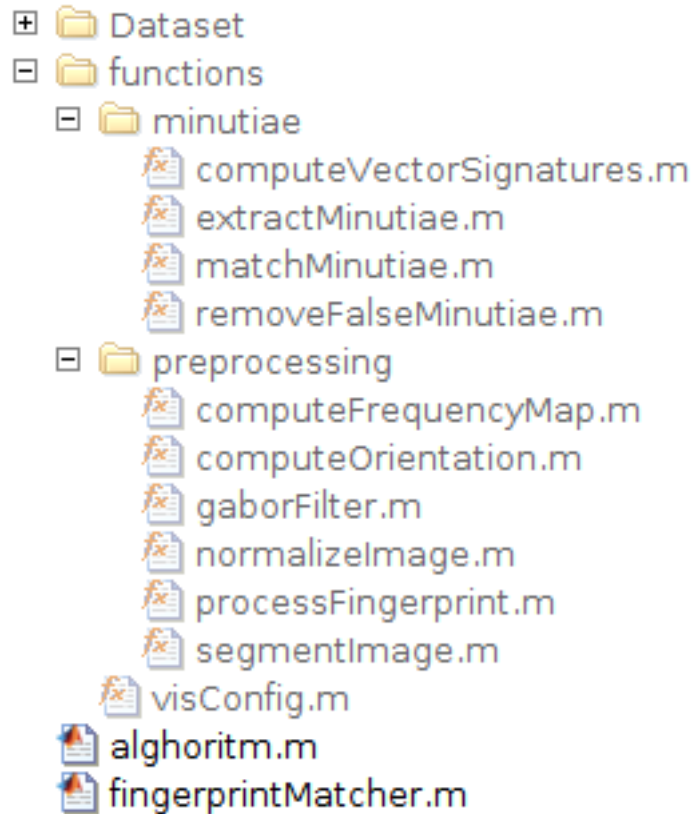
3.1 Stack technologiczny

W projekcie wykorzystano następujący stack technologiczny:

- **Język programowania:** MATLAB (skrypty i funkcje .m).
- **Wymagane pakiety (Toolboxy):**
 - **Image Processing Toolbox:** Niezbędny do operacji takich jak wczytywanie obrazów, binaryzacja, szkieletyzacja (bwmorph), filtracja (imfilter) oraz transformacje morfologiczne.
 - **Signal Processing Toolbox:** Wykorzystywany w standardowej wersji algorytmu do analizy częstotliwości (funkcje findpeaks, xcorr), choć projekt dopuszcza implementację zamienników w podstawowym środowisku MATLAB.
- **Dane wejściowe:** Obrazy odcisków palców w formacie .tif, przechowywane w dedykowanym folderze Dataset.

3.2 Architektura systemu

Projekt został zorganizowany w sposób **modularny**, co pozwala na niezależne testowanie każdego etapu przetwarzania obrazu.

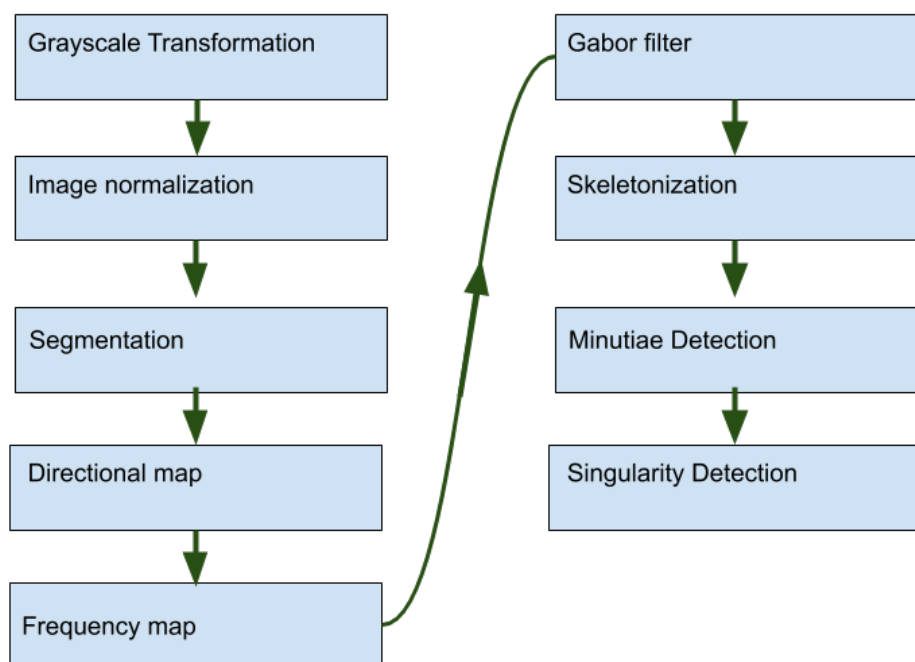


Rysunek 1: Struktura projektu

- Dataset/ – folder zawierający obrazy testowe odcisków palców w formacie .tif.
- alghoritm.m – główny skrypt sterujący, integrujący wszystkie etapy algorytmu.
- **Moduły Przetwarzania:**
 - normalizeImage.m, segmentImage.m – przygotowanie obrazu.
 - computeOrientation.m, computeFrequencyMap.m – analiza cech polowych.
 - gaborFilter.m – wzmacnianie struktury linii.
 - extractMinutiae.m, removeFalseMinutiae.m – praca z punktami charakterystycznymi.
 - computeVectorSignatures.m – generowanie sygnatury do porównania.

4 Analiza etapów algorytmu

Algorytm rozpoznawania odcisków palców opiera się na **łańcuchu przetwarzania**, gdzie wynik jednego etapu stanowi wejście dla kolejnego.



Rysunek 2: Algorytm przetwarzania

Algorytm można podzielić na trzy główne warstwy:

1. **Warstwa przetwarzania wstępnego (Pre-processing)**

Jest niezwykle istotną fazą algorytmu. Pozwala ono na poprawę jakości obrazu i usunięcie szumów, co ułatwia realizację zadań w drugim kroku oraz optymalizuje proces przetwarzania.

2. **Warstwa ekstrakcji cech (Feature Extraction)**

Głównym celem tej warstwy jest transformacja reprezentacji rastrowej na postać wektorową, czyli wyodrębnienie matematycznego opisu unikalnych cech biologicznych.

3. **Warstwa Post-processingu i Sygnatury**

Ta warstwa stanowi krytyczny etap weryfikacji danych, mający na celu odseparowanie rzeczywistych cech biometrycznych od błędów algorytmicznych.

4.1 Warstwa przetwarzania wstępnego (Pre-processing)

Jak zostało wspomniane wcześniej, ta warstwa jest kluczowa dla usunięcia szumów i przygotowania wyraźnej struktury linii papilarnych do dalszej analizy i składa się z następujących etapów.

4.1.1 Konwersja do skali szarości

Do analizy linii papilarnych informacje o kolorze są zbędne, dlatego obraz konwertowany jest na 8-bitową skalę szarości. Krok ten pozwala zoptymalizować ogólny wygląd obrazu i ułatwia przetwarzanie biometryczne.

W implementacji sprawdzany format wejściowy obrazu – jeśli jest on **kolorowy (RGB)**, następuje **konwersja do skali szarości**, a następnie rzutowanie na typ zmiennoprzecinkowy `double` w celu zachowania precyzji obliczeń.

```
1 I_origin = double(I_origin);
2 if ndims(I_origin) == 3
3     I_gray = rgb2gray(I_origin);
4 else
5     I_gray = I_origin;
6 end
7 I_gray = double(I_gray);
```

4.1.2 Normalizacja

Normalizacja ujednolica jasność i poprawia kontrast obrazu. Jej głównym celem jest redukcja wariancji wzdłuż linii papilarnych, co ułatwia dalszą obróbkę. Proces ten realizowany jest w funkcji `normalizeImage` i odbywa się lokalnie w każdym bloku według następujących kroków:

1. Obliczanie średniej

Średnia wartość jasności obrazu M o wymiarach $n \times m$ jest obliczana jako suma wszystkich pikseli podzielona przez ich liczbę:

$$M = \frac{1}{n \times m} \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} I(i, j) \quad (1)$$

W kodzie odpowiada za to funkcja `mean2`:

```
1
2 M = mean2(I_gray);
3
```

2. Obliczanie wariancji

Wariancja V określa rozrzut wartości poziomów szarości wokół średniej:

$$V = \frac{1}{n \times m} \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} (I(i, j) - M)^2 \quad (2)$$

W programie wykorzystano funkcję `var` na spłaszczonym wektorze pikseli:

```
1
2 V = var(I_gray(:));
3
```

3. Obliczanie znormalizowanej jasności piksela $N(i, j)$

Finalny wzór na normalizację, który dostosowuje jasność każdego piksela do zadanej średniej M_0 i wariancji V_0 , jest funkcją warunkową:

$$N(i, j) = \begin{cases} M_0 + \sqrt{\frac{V_0 \times (I(i, j) - M)^2}{V}} & \text{jeśli } I(i, j) > M \\ M_0 - \sqrt{\frac{V_0 \times (I(i, j) - M)^2}{V}} & \text{jeśli } I(i, j) \leq M \end{cases} \quad (3)$$

W kodzie zaimplementowano to w sposób wektorowy przy użyciu masek logicznych `mask1` i `mask2`, co znacząco przyspiesza obliczenia:

```
1 mask1 = I_norm > M;  
2 mask2 = I_norm <= M;  
3  
4 I_norm(mask1) = M0 + sqrt(V0 * (M - I_norm(mask1)).^2 / V);  
5 I_norm(mask2) = M0 - sqrt(V0 * (M - I_norm(mask2)).^2 / V);  
6
```

Normalizacja nie zmienia struktury obrazu, natomiast służy do standaryzacji zmienności poziomów szarości.

4.1.3 Segmentacja

Celem segmentacji jest oddzielenie istotnych danych biometrycznych od tła i szumów. Segmentacja jest niezbędnym etapem pozwalającym na wyeliminowanie krawędzi obrazu oraz obszarów o zbyt wysokim poziomie zaszumienia, które mogłyby generować fałszywe minucje. Proces ten opiera się na analizie wariancji poziomów szarości w lokalnych blokach obrazu.

Zasada działania algorytmu:

- **Podział na bloki:** Obraz jest dzielony na podbloki o rozmiarze $W \times W$. W implementacji przyjęto $W = 16$, co pozwala na stabilną estymację parametrów statystycznych.
- **Obliczanie wariancji:** Dla każdego bloku obliczana jest wariancja zgodnie ze wzorem (2), co pozwala ocenić dynamikę zmian jasności w danym obszarze, czy dany fragment zawiera istotne informacje (prążki), czy jedynie szum tła.
- **Kryterium decyzyjne:** Pierwiastek z wariancji każdego bloku jest porównywany z progiem T . W kodzie zaimplementowano to jako porównanie wariancji z kwadratem progu ($V \geq T^2$), co jest matematycznie równoważne, ale bardziej wydajne obliczeniowo.
 - Jeśli wartość jest niższa od progu, blok jest klasyfikowany jako tło i wykluczany z dalszej obróbki.
 - Jeśli wartość przekracza próg, blok uznaje się za część użyteczną obrazu.

Implementacja algorytmu:

```
1 for i = 1:block_size:nRows-block_size+1
2     for j = 1:block_size:nCols-block_size+1
3         block = I_norm(i:i+block_size-1, j:j+block_size-1);
4
5         % Sprawdzenie kryterium wariancji wzgledem progu T=5
6         if var(double(block(:))) >= T*T
7             mask(i:i+block_size-1, j:j+block_size-1) = true;
8         end
9     end
10 end
```

4.1.4 Analiza polowa

Kolejnym kluczowym krokiem w przetwarzaniu wstępnym jest analiza cech polowych, która obejmuje wyznaczenie lokalnej orientacji linii papilarnych oraz ich częstotliwości. Etap ten jest niezbędny do poprawnego działania filtracji Gabora, która adaptacyjnie wzmacnia strukturę obrazu w oparciu o te parametry.

1. **Wyznaczanie mapy kierunkowej** Mapa kierunkowa definiuje lokalny kierunek przebiegu linii papilarnych (grzbietów) w danym punkcie obrazu. Proces ten składa się z dwóch głównych etapów: estymacji orientacji oraz jej wygładzania.]

- **Estymacja orientacji**

Orientacja nie może być wyznaczona bezpośrednio z wartości jasności pojedynczego piksela. W tym celu analizuje się otoczenie V o rozmiarze $W \times W$ pikseli i oblicza gradienty składowe G_x oraz G_y przy użyciu masek Sobela:

$$G_x = V(x, y) * \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (4)$$

$$G_y = V(x, y) * \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (5)$$

W implementacji wykorzystano funkcję `imgradientxy` z maskami Sobela, co odpowiada spłotowi obrazu z macierzami gradientu G_x i G_y .

```
1 % Wstepne wygładzenie i obliczenie gradientow Gx i Gy
2 I_smooth = imgaussfilt(I_seg, 0.5);
3 [Gx, Gy] = imgradientxy(I_smooth, 'sobel');
```

Następnie wyznacza się lokalny kierunek w otoczeniu piksela (i, j) poprzez obliczenie sumy gradientów:

$$Vx(i, j) = \sum_{u=i-\frac{W}{2}}^{i+\frac{W}{2}} \sum_{v=j-\frac{W}{2}}^{j+\frac{W}{2}} (2 \cdot G_x(u, v) \cdot G_y(u, v)) \quad (6)$$

$$Vy(i, j) = \sum_{u=i-\frac{W}{2}}^{i+\frac{W}{2}} \sum_{v=j-\frac{W}{2}}^{j+\frac{W}{2}} (Gx(u, v)^2 - Gy(u, v)^2) \quad (7)$$

W kodzie sumowanie blokowe zastąpiono splotem z filtrem Gaussa o odchyleniu $\sigma = W/2$, co zapewnia płynniejsze przejścia między obszarami:

```
1 % Obliczanie składników sumy gradientów
2 Gxx_smooth = imfilter(Gx.^2, h, 'symmetric');
3 Gyy_smooth = imfilter(Gy.^2, h, 'symmetric');
4 Gxy_smooth = imfilter(Gx .* Gy, h, 'symmetric');
5
6 % Wyznaczenie Vx i Vy zgodnie ze wzorami (6) i (7)
7 Vx = 2 * Gxy_smooth;
8 Vy = Gxx_smooth - Gyy_smooth;
```

Wstępna orientacja $\theta(i, j)$ jest obliczana jako:

$$\theta(i, j) = \frac{1}{2} \tan^{-1} \frac{Vx(i, j)}{Vy(i, j)} \quad (8)$$

```
1 % Obliczenie kata theta (Wzor 8)
2 theta = 0.5 * atan2(Vx, Vy + eps);
3
```

- Wygładzanie mapy kierunkowej

Ze względu na obecność szumu, wstępna mapa orientacji może zawierać nieciągłości. Aby je wyeliminować, orientacja θ jest przekształcana na ciągłe pole wektorowe (ϕ_x, ϕ_y) , poddawane filtracji dolnoprzepustowej.

$$\phi_x(i, j) = \cos(2 \cdot \theta(i, j)) \quad (9)$$

$$\phi_y(i, j) = \sin(2 \cdot \theta(i, j)) \quad (10)$$

Wygładzone składowe (ϕ'_x, ϕ'_y) wyznacza się z użyciem filtra Gaussa G o rozmiarze $W_\phi \times W_\phi$

$$\phi'_x(i, j) = \sum_{u=-\frac{W_\phi}{2}}^{\frac{W_\phi}{2}} \sum_{v=-\frac{W_\phi}{2}}^{\frac{W_\phi}{2}} G(u, v) \cdot \phi_x(i - u \cdot w, j - v \cdot w) \quad (11)$$

$$\phi'_y(i, j) = \sum_{u=-\frac{W_\phi}{2}}^{\frac{W_\phi}{2}} \sum_{v=-\frac{W_\phi}{2}}^{\frac{W_\phi}{2}} G(u, v) \cdot \phi_y(i - u \cdot w, j - v \cdot w) \quad (12)$$

```
1 % Konwersja do składowych Phi_x i Phi_y (Wzory 10-11)
2 Phi_x = cos(2 * theta);
3 Phi_y = sin(2 * theta);
4
5 % Wygładzanie składowych filtrem Gaussa (Wzory 12-13)
6 Phi_x_smooth = imfilter(Phi_x, msk_smooth, 'symmetric');
7 Phi_y_smooth = imfilter(Phi_y, msk_smooth, 'symmetric');
```

Finalna mapa orientacji $O(i, j)$ jest wyznaczana poprzez powrót z dziedziny wektorowej do kątowej. Dodatkowo w kodzie zastosowano korekcję o kąt $\pi/2$, aby wektory były skierowane wzdłuż prążków (równolegle), a nie wzdłuż gradientu (prostopadle).

$$O(i, j) = \frac{1}{2} \tan^{-1} \frac{\phi'y(i, j)}{\phi'x(i, j)} \quad (13)$$

```
1 % Obliczenie wygładzonej orientacji (Wzór 13)
2 theta_map = 0.5 * atan2(Phi_y_smooth, Phi_x_smooth);
3
4 % Korekcja kąta w celu wyrownania z przebiegiem prążków
5 theta_map = theta_map + pi/2;
```

2. Wyznaczanie mapy częstotliwości Mapa częstotliwości określa lokalną gęstość występowania grzbietów w obrazie. Jest ona kluczowym parametrem dla adaptacyjnej filtracji, ponieważ pozwala dostosować rozmiar jądra filtra do rzeczywistej odległości między prążkami, która jest cechą osobniczą.

- **Projekcja sygnatury prążków**

W celu wyznaczenia częstotliwości w konkretnym bloku, jest on rotowany zgodnie z lokalną orientacją tak, aby prążki były ustawione pionowo. Następnie wyznaczana jest 1-wymiarowa sygnatura V poprzez sumowanie wartości pikseli wzdłuż wierszy.

```
1 % Rotacja bloku prostopadle do orientacji lokalnej
2 rotated_block = imrotate(block, rad2deg(dir_rad), 'bilinear', '
    crop');
3
4 % Obliczenie wektora rzutu (projekcja ortogonalna)
5 V = sum(rotated_block, 2);
6 V = V - mean(V); % Usunięcie składowej stałej
```

- **Estymacja przestrzenna częstotliwości**

Na podstawie uzyskanego wektora ekstremów oblicza się **średnią odległość** między kolejnymi prążkami, oznaczaną jako $S(i, j)$. Do dokładnego wyznaczenia tej odległości (długości fali) wykorzystywana jest autokorelacja wektora V (`xcorr`) oraz detekcja szczytów (`findpeaks`).

Finalna częstotliwość $F(i, j)$ dla bloku (i, j) jest obliczana według wzoru:

$$F(i, j) = \frac{1}{S(i, j)} \quad (14)$$

- Jeśli odległość między maksimum a minimum jest mniejsza niż zadany próg, uznaje się to za szum i eliminuje z obliczeń.
- Jeśli blok nie zawiera co najmniej dwóch maksimum, okres przyjmuje wartość zero, co oznacza tło lub zbyt wysoki poziom szumu.

```
1 % Obliczanie autokorelacji wektora V
2 ac = xcorr(V, 'coeff');
3 center_idx = ceil(length(ac)/2);
```

```

4 ac = ac(center_idx:end); % Analiza prawej strony symetrycznej
    funkcji
5
6 % Detekcja szczytów
7 [pks, locs] = findpeaks(ac);
8 if ~isempty(locs)
9     % Znalezienie pierwszego szczytu powyżej minimalnej długości
    fali
10     valid_peak_idx = find(locs > minWL, 1);
11     if ~isempty(valid_peak_idx)
12         waveLength = locs(valid_peak_idx) - 1;
13         freq = 1 / waveLength;
14     end
15 end

```

- **Konstrukcja i wygładzanie mapy globalnej**

Po obliczeniu częstotliwości dla wszystkich bloków, mapa poddawana jest procesowi **uzupełniania dziur oraz wygładzania**. Dzięki temu filtr Gabora nie generuje artefaktów na granicach bloków.

```

1 % Wypełnianie dziur mediana globalna
2 valid_vals = freq_map_block(freq_map_block > 0);
3 global_median = median(valid_vals);
4 freq_map_block(freq_map_block == 0 & mask_block) = global_median;
5
6 % Wygładzanie filtrem Gaussa dla zapewnienia płynności zmian
7 msk = fspecial('gaussian', 7, 1.5);
8 freq_smooth = imfilter(freq_map_block, msk, 'same');
9
10 % Skalowanie do oryginalnego rozmiaru obrazu
11 freq_map = imresize(freq_smooth, [nRows, nCols], 'bilinear');

```

Finalna częstotliwość $F(i, j)$ jest ograniczona parametrami `minWaveLength` (7) oraz `maxWaveLength` (15), co zapobiega interpretacji szumu o bardzo wysokiej lub niskiej częstotliwości jako struktury linii papilarnych.

4.1.5 Wzmocnienie

Filtracja Gabora jest "sercem" algorytmu **wzmacniania obrazu**, ponieważ łączy wszystkie informacje zebrane w poprzednich krokach (normalizację, mapę orientacji i częstotliwości), aby wygenerować czysty i wyraźny obraz odcisku palca.

Zasada filtrowania polega na modyfikacji wartości pikseli obrazu, zazwyczaj w celu poprawy jego wyglądu i sprowadza się to do utworzenia nowego obrazu przy użyciu wartości pikseli obrazu oryginalnego, aby wyselekcjonować w dziedzinie Fouriera zestaw częstotliwości, które tworzą obszar przeznaczony do wykrycia. Zastosowany filtr to filtr Gabora o symetrii parzystej, zorientowany pod kątem 0 stopni:

$$h_b(x_\theta, y_\theta; \theta, f) = \exp \left\{ -\frac{1}{2} \left(\frac{x_\theta^2}{\sigma_x^2} + \frac{y_\theta^2}{\sigma_y^2} \right) \right\} \cdot \cos(2\pi f x_\theta) \quad (15)$$

Wartości σ_x oraz σ_y są dobierane w taki sposób, aby $\sigma_x = k_x \cdot F(i, j)$ oraz $\sigma_y = k_y \cdot F(i, j)$, gdzie wartości k_x i k_y są ustalone na poziomie 0,5.

Aby uzyskać inne orientacje, wystarczy dokonać rotacji osi układu współrzędnych zgodnie ze wzorem:

$$\begin{bmatrix} x_\theta \\ y_\theta \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \quad (16)$$

Zależnie od różnych bloków obrazu, filtr może mieć kilka preferowanych kierunków. W takim przypadku filtr końcowy jest sumą filtrów podstawowych umieszczonych w każdym kierunku. Wynikowy obraz jest splotem przestrzennym obrazu oryginalnego (znormalizowanego) oraz jednego z filtrów bazowych w kierunku i lokalnej częstotliwości pochodzących z map kierunkowej i częstotliwościowej, zgodnie ze wzorem:

$$E(i, j) = \sum_{u=-\frac{w_x}{2}}^{\frac{w_x}{2}} \sum_{v=-\frac{w_y}{2}}^{\frac{w_y}{2}} h_b(u, v; O(i, j), F(i, j)) \cdot N(i - u, j - v) \quad (17)$$

Gdzie: $E(i, j)$ to nowa wartość piksela (i, j) , $O(i, j)$ i $F(i, j)$ to wartości pikseli (i, j) odpowiednio z mapy orientacji i częstotliwości, w_x i w_y to odpowiednio długość i szerokość bloku użytego do splotu.

Implementacja

- Budowa jądra filtra Gabora

W implementacji **jądro filtra** jest generowane dynamicznie dla każdego piksela (lub bloku) w oparciu o lokalne parametry pola. Kluczowym elementem jest **rotacja układu współrzędnych**, która pozwala ustawić filtr zgodnie z lokalnym kierunkiem prążków $O(i, j)$.

```

1 % Rotacja osi zgodnie z lokalna orientacja (Wzor 16)
2 x_prime = x * cos(orient_val) + y * sin(orient_val);
3 y_prime = -x * sin(orient_val) + y * cos(orient_val);
4
5 % Definicja parzystego filtra Gabora (Wzor 15)
6 G = exp(-((x_prime.^2)/(2*sigma_x^2) + (y_prime.^2)/(2*sigma_y^2))) .*
    cos(2*pi*f*x_prime);
7
8 % Usuniecie składowej stałej (DC offset)
9 G = G - mean(G(:));

```

- Adaptacja parametrów σ_x i σ_y

Zgodnie z teorią, parametry odchylenia standardowego obwiedni Gaussa powinny być skorelowane z częstotliwością prążków. W kodzie zastosowano parametr `sigmaratio` (ustalony na 0.5), który dostosowuje szerokość filtra do długości fali $(1/f)$.

```

1 wavelength = 1/f;
2 sigma_x = sigma_ratio * wavelength; % kx * F(i,j)
3 sigma_y = sigma_y_const;           % Stala szerokosc filtra
4

```

- **Splot przestrzenny** Finalny obraz $E(i, j)$ powstaje poprzez splot znormalizowanego obrazu wejściowego z przygotowanym jądrem G . Operacja ta jest wykonywana jedynie wewnątrz obszaru maski, co optymalizuje czas obliczeń i zapobiega powstawaniu artefaktów na brzegach tła.

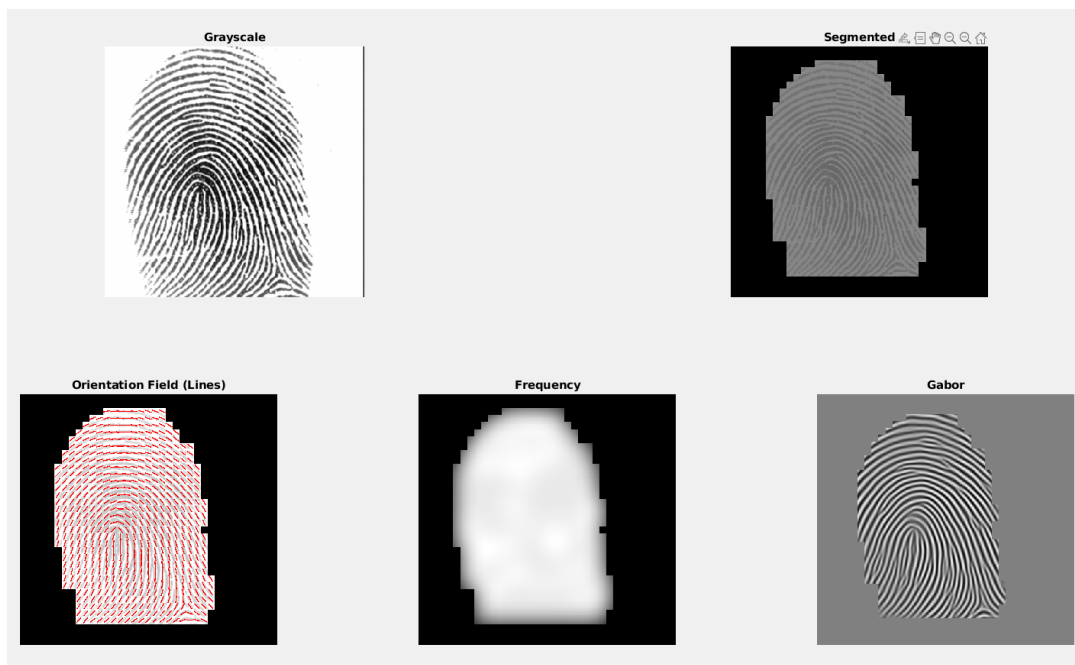
```

1      % Pobranie otoczenia piksela o rozmiarze K x K
2      area = I_norm(r-halfK : r+halfK, c-halfK : c+halfK);
3
4      % Obliczenie nowej wartosci piksela (Suma iloczynow)
5      I_enhanced(r, c) = sum(sum(double(area) .* G));
6

```

4.1.6 Wyniki przetwarzania wstępnego

Zdjęcie podsumowujące przedstawia wizualizację kluczowych etapów przetwarzania wstępnego, od surowego obrazu wejściowego aż po wynik końcowej filtracji.



Rysunek 3: Wyniki przetwarzania wstępnego

Opis etapów przetwarzania:

- **Grayscale (Skala szarości):** Jest to obraz wejściowy po konwersji z formatu kolorowego (RGB) na 8-bitową skalę szarości. Stanowi on podstawę do wszystkich dalszych obliczeń. Na tym etapie widoczne są jeszcze szumy oraz nieistotne tło wokół palca.
- **Segmented (Segmentacja):** Obraz po zastosowaniu funkcji `segmentImage.m`. Algorytm skutecznie odizolował obszar zawierający linie papilarne od czarnego tła. Dzięki temu system nie traci czasu na obliczenia w miejscach, gdzie nie ma danych biometrycznych.

- **Orientation Field (Mapa orientacji):** Wizualizacja wyniku funkcji `computeOrientation.m`. Nałożone czerwone linie reprezentują lokalny kierunek grzbietów w każdym bloku obrazu. Jak widać, linie te idealnie podążają za krzywizną odcisków, co jest niezbędne do poprawnego ustawienia filtra Gabora.
- **Frequency (Mapa częstotliwości):** Reprezentacja gęstości linii papilarnych wygenerowana przez `computeFrequencyMap.m`. Jasne i ciemne obszary (rozmyta plama) pokazują, jak zmienia się odległość między grzbietami w różnych częściach palca. Jest to parametr sterujący "szerokością" pasm filtra Gabora.
- **Gabor (Wynik filtracji):** Finalny produkt warstwy przetwarzania wstępnego, uzyskany dzięki `gaborFilter.m`. W porównaniu do pierwszego panelu (Grayscale), linie są tutaj znacznie wyraźniejsze, przerwy zostały scalone, a szum tła całkowicie usunięty. Obraz ten jest gotowy do binaryzacji i ekstrakcji minucji.

4.2 Warstwa ekstrakcji cech

Celem warstwy ekstrakcji cech jest wyodrębnienie z oczyszczonego szkieletu obrazu unikalnych punktów charakterystycznych, zwanych **minucjami**. To właśnie te punkty tworzą niepowtarzalną "sygnaturę" biometryczną, która pozwala na późniejszą identyfikację osoby.

4.2.1 Binarizacja obrazu

Aby umożliwić szkieletyzację, obraz musi zostać najpierw przekształcony z 256 poziomów szarości na obraz binarny (dwuwartościowy). W tym procesie czarne piksele odpowiadają grzbietom (streaks), a białe dolinom (valleys).

- **Metoda:** Zastosowano prostą i skuteczną metodę progowania, gdzie wartość każdego piksela $P(x, y)$ jest porównywana z progiem M .
- **Logika:** Jeśli wartość piksela jest większa od progu, przyjmuje on wartość 1 (czarny), w przeciwnym razie 0 (biały).
- **W kodzie:** Funkcja realizuje to zadanie, przyjmując 50 procent progu jasności dla wzmocnionego obrazu Gabora.

```

1 I_{binary} = imbinarize(I_{enhanced}, 0.5)
2

```

4.2.2 Szkieletyzacja obrazu

Szkieletyzacja jest niezbędna do ułatwienia ekstrakcji minucji. Polega ona na sekwencyjnym stosowaniu operacji morfologicznej erozji, która redukuje grubość linii papilarnych, aż osiągnie ona szerokość dokładnie jednego piksela.

- **Zasada:** Podczas odchudzania linii musi zostać zachowana ich ciągłość (connectivity) – algorytm nie może tworzyć przerw ani dziur w istniejących liniach.

- W kodzie:

```
1 I_{skel} = bwmorph(~I_{binary}, 'skel', Inf)
2
```

– wykonuje szkieletyzację na odwróconym obrazie (aby operować na grzbietach).

```
1 I_{skel} = bwmorph(I_{skel}, 'spur', 5)
2
```

– dodatkowo usuwa tzw. "ostrogi", czyli krótkie, błędne odgałęzienia, które mogłyby zostać uznane za fałszywe minucje.

```
1 I_{skel}(~mask)=0
2
```

– końcowe czyszczenie tła przy użyciu wcześniej wyznaczonej maski.

4.2.3 Wyniki binaryzacji i szkieletyzacji obrazu

Na podstawie przeprowadzonych operacji morfologicznych uzyskano następujące rezultaty.



Rysunek 4: Wyniki binaryzacji i szkieletyzacji obrazu

- **Binary:** Obraz czarno-biały, gdzie grzbiety są czarne, a doliny i tło białe. Tutaj zniknął już wszelki szum szarości widoczny po filtrze Gabora.
- **Skelet:** Wynik szkieletyzacji. Każda linia papilarna została zredukowana do grubości jednego piksela. To właśnie na tym obrazie Twój algorytm będzie szukał końców i rozgałęzień linii, ponieważ ich znalezienie na tak cienkich nitkach jest matematycznie bardzo proste.

4.3 Warstwa Post-processingu i Sygnatury

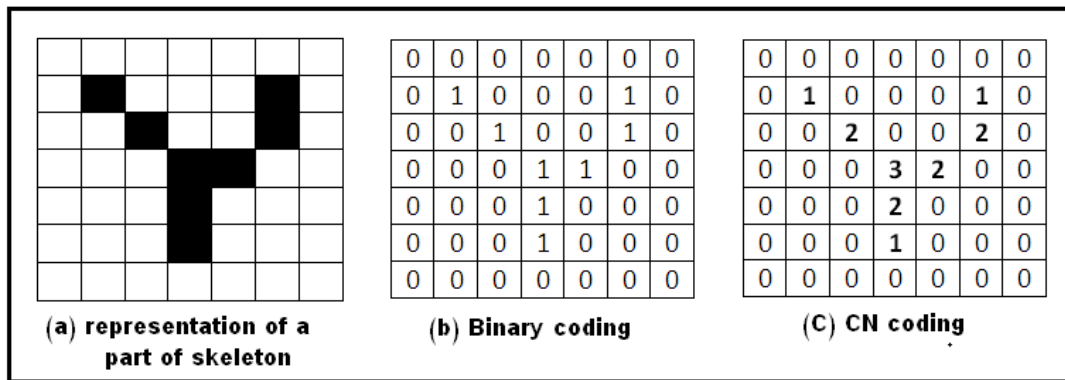
Ostatnim etapem przygotowania danych do końcowego dopasowania jest warstwa post-processingu oraz wygenerowanie ostatecznej sygnatury biometrycznej. Jest to krytyczny moment algorytmu, ponieważ surowa ekstrakcja punktów charakterystycznych często obarczona jest błędami wynikającymi z niedoskonałości obrazu wejściowego.

4.3.1 Detekcji minucji

Crossing Number (CN) to najpowszechniej stosowana metoda detekcji minucji ze względu na jej prostotę obliczeniową. Pozwala ona na matematyczne określenie typu punktu charakterystycznego poprzez analizę jego bezpośredniego otoczenia.

1. Zasada działania

Algorytm operuje na obrazie po szkieletyzacji, gdzie piksele grzbietów mają wartość 1, a piksele tła wartość 0. Dla każdego piksela grzbietu P analizowane jest jego otoczenie w oknie 3×3 (8 sąsiadów).



Rysunek 5: Różne reprezentacje szkieletów

2. Wzór matematyczny

Wartość CN dla piksela P obliczana jest według następującej formuły:

$$CN(P) = \frac{1}{2} \sum_{i=1}^8 |P_i - P_{i-1}| \quad (18)$$

Gdzie P_i to wartości sąsiednich pikseli w sekwencji wokół punktu centralnego ($P_8 = P_0$). U nas w kodzie realizuje to linia:

```
1 cn = 0.5 * sum(abs(diff(P)))
2
```

3. Interpretacja wyników (Klasyfikacja minucji)

W zależności od uzyskanej wartości liczbowej, punkt P jest klasyfikowany w następujący sposób:

Tabela 2: Klasyfikacja minucji na podstawie wartości Crossing Number (CN)

Wartość CN	Typ punktu	Znaczenie w algorytmie
CN = 0	Punkt izolowany	Ignorowany (zazwyczaj szum)
CN = 1	Zakończenie (Termination)	Minucja – koniec linii papilarnej
CN = 2	Punkt kontynuacji	Zwykły fragment linii (brak minucji)
CN = 3	Rozwidlenie (Bifurcation)	Minucja – rozdzielenie się linii na dwie
CN = 4	Rozwidlenie poczwórne	Bardzo rzadkie, zazwyczaj błąd/szum

4. Implementacja

W funkcji `extractMinutiae.m` algorytm przeszukuje obraz piksel po pikselu i sprawdza warunki:

- Jeśli `cn == 1`, współrzędne (i, j) są zapisywane w tabeli terminations.
- Jeśli `cn == 3`, współrzędne (i, j) trafiają do tabeli bifurcations.

4.3.2 Eliminacja fałszywych minucji

Etap ten jest krytyczny dla końcowej skuteczności systemu, ponieważ pozwala na wyselekcjonowanie jedynie prawdziwych cech charakterystycznych, które posłużą do zdefiniowania wektora sygnatury odcisku palca.

Proces eliminacji polega na **analizie punktów**, dla których wskaźnik Crossing Number (CN) wynosi 1 (zakończenia) lub 3 (bifurkacje).

Zastosowany algorytm opiera się na wynikach empirycznych, które wskazują, że odległość między dwiema sąsiednimi, prawdziwymi minucjami jest prawie zawsze większa od pewnego progu. Jako próg odległości przyjęto wartość $K1 = 9$ pikseli, co odpowiada średniej odległości między prążkami:

1. Przetwarzanie wykrytych zakończeń

W przypadku wykrycia kandydata na zakończenie $T(T_x, T_y)$, algorytm wykonuje następujące kroki:

- **Filtracja krawędziowa:** W pierwszej kolejności sprawdzane jest położenie punktu względem granic obrazu oraz tła. Ponieważ szkieletyzacja prążków urywających się na granicy obszaru segmentacji generuje mnóstwo błędnych zakończeń, punkty te są automatycznie odrzucane. Jeśli zakończenie T znajduje się w bloku przylegającym do tła obrazu, jest uznawane za fałszywe.

Implementacja: na samym początku funkcji usuwamy punkty, które nie mieszczą się w masce (obszarze odcisku):

```

1 if ~isempty(terms)
2     idx_t = diag(mask(terms(:,1), terms(:,2)));
3     terms = terms(idx_t == 1, :);
4 end
5

```

- **Analiza dystansu:** Dla pozostałych punktów analizowany jest prążek, do którego należą, na dystansie maksymalnie $K1$. Wyróżnia się dwa główne przypadki błędów:

- **Gałąź pasożytnicza:** Jeśli przed osiągnięciem dystansu $K1$ napotkany zostanie punkt A , który jest bifurkacją ($CN(A) = 3$), uznaje się, że punkt T jest zakończeniem błędu strukturalnego. Wówczas zarówno punkt T , jak i bifurkacja A są usuwane.

Implementacja:

```

1 for i = 1:size(terms, 1)
2     for j = 1:size(bifs, 1)
3         % Odleglosc miedzy T a B mniejsza niz K1
4         if norm(terms(i,:) - bifs(j,:)) < K1
5             valid_terms(i) = false; % Usun zakonczenie
6             valid_bifs(j) = false;  % Usun bifurkacje
7         end
8     end
9 end
10

```

- **Krótki segment:** Jeśli przed osiągnięciem dystansu $K1$ napotkane zostanie inne zakończenie ($CN(A) = 1$), oznacza to, że mamy do czynienia z izolowanym fragmentem prążka lub szumem. Oba punkty są eliminowane.

Implementacja:

```

1 for i = 1:size(terms, 1)
2     for j = i+1:size(terms, 1)
3         % Dwa zakonczenia zbyt blisko siebie
4         if norm(terms(i,:) - terms(j,:)) < K1
5             valid_terms(i) = false;
6             valid_terms(j) = false;
7         end
8     end
9 end
10

```

2. Przetwarzanie wykrytych bifurkacji

Dla każdego punktu bifurkacji B analizowane są trzy wychodzące z niego odnogi prążków na maksymalnym dystansie $K1$. Proces filtracji przebiega według następującej hierarchii:

- **Strefa zaszumiona:** Jeśli wszystkie trzy odnogi kończą się (osiągają inne minucje) na dystansie mniejszym niż $K1$, oznacza to lokalne nagromadzenie szumu. W obszarze o promieniu mniejszym niż średnia odległość między prążkami znajduje się zbyt wiele punktów, co jest sprzeczne z naturalną topologią odcisku. Punkt B zostaje odrzucony.

Impementacja: realizujemy to poprzez usuwanie bifurkacji, które znajdują się zbyt blisko siebie, co jest najczęstszym objawem „strefy zaszumionej”

```

1 for i = 1:size(bifs, 1)
2     for j = i+1:size(bifs, 1)
3         % Sprawdzenie czy bifurkacje sa w "strefie szumu"
4         if norm(bifs(i,:) - bifs(j,:)) < K1
5             valid_bifs(i) = false; % Odrzucenie punktu B
6             valid_bifs(j) = false;
7         end
8     end
9 end
10

```

- **Bifurkacja z zakończeniem:** Jeśli przynajmniej jedna z trzech odnóg prowadzi do zakończenia ($CN = 1$) na dystansie $d \leq K1$, uznaje się, że bifurkacja jest częścią błędu (np. mostka, który się nie domknął). W tej sytuacji zarówno bifurkacja B , jak i napotkane zakończenie są usuwane.

Implementacja: wykrywa sytuację, w której zakończenie prążka ($CN = 1$) jest zbyt blisko rozwidlenia ($CN = 3$):

```

1 for i = 1:size(terms, 1)
2     for j = 1:size(bifs, 1)
3         % Jesli branch prowadzi do zakonczenia d < K1
4         if norm(terms(i,:) - bifs(j,:)) < K1
5             valid_terms(i) = false; % Odrzucenie zakonczenia
6             valid_bifs(j) = false; % Odrzucenie bifurkacji
7         end
8     end
9 end
10

```

- **Wyspa:** Jeśli dwie z trzech odnóg bifurkacji prowadzą do tego samego punktu (zamykają się w pętlę), mamy do czynienia z tzw. wyspą. Zgodnie z definicją topologii prążków, prawdziwa bifurkacja reprezentuje rozwidlenie pojedynczego prążka na dwa oddzielne, dlatego struktury typu "wyspa" są eliminowane.

Implementacja: usuwamy takie struktury, wykrywając mały dystans między punktami rozwidlenia:

```

1 if norm(bifs(i,:) - bifs(j,:)) < K1
2     valid_bifs(i) = false;
3     valid_bifs(j) = false;
4 end
5

```

Wszystkie punkty, które nie zostały odrzucone w powyższych krokach, są z twierdzane jako prawdziwe minucje i przekazywane do etapu tworzenia wektora sygnatury.

4.3.3 Sygnatura wektorowa

Celem tego etapu jest sformułowanie ostatecznego wektora cech, który posłuży do porównywania odcisków. Aby algorytm był odporny na przesunięcia (translacje) oraz obroty (rotacje) palca na czytniku, zastosowano lokalną reprezentację sąsiedztwa. Zamiast polegać na bezwzględnych współrzędnych (x, y) , każda minucja opisana

jest przez relację z K jej najbliższymi sąsiadami. Dzięki temu, nawet jeśli cały odcisk zostanie przesunięty, relatywne odległości i kąty między minucjami pozostaną stałe (niezmienniczość geometryczna).

Implementacja algorytmu:

1. **Wybór lokalnego sąsiedztwa** Dla każdej minucji centralnej m_i algorytm wyznacza K najbliższych sąsiadów m_j na podstawie odległości euklidesowej.

```

1 % Fragment kodu: Wybór K-sasiadow
2 % Obliczanie dystansu euklidesowego
3 dist = sqrt((mi(1)-mj(1))^2 + (mi(2)-mj(2))^2);
4 distances = sortrows(distances, 2); % Sortowanie od najbliższych
5 num_neighbors = min(K, size(distances, 1)); % Wybór K-najbliższych
   punktow
6
```

2. **Wyznaczanie cech relatywnych** Dla każdego sąsiada w wektorze sygnatury obliczane są cztery parametry:

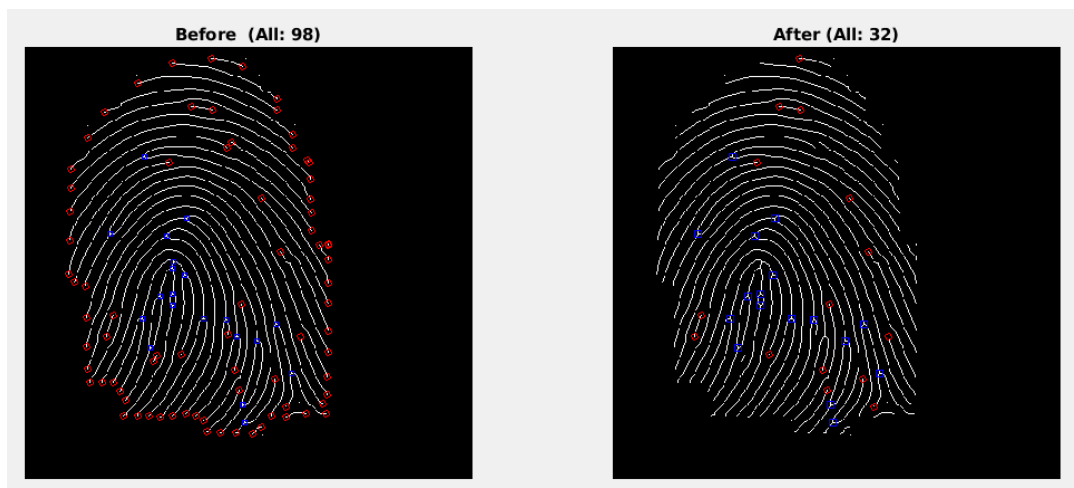
- d_{ij} : Odległość między minucją centralną a sąsiadem.
- α_{ij} : Względna różnica orientacji prążków.
- β_{ij} : Kąt kierunkowy sąsiada względem orientacji minucji centralnej.
- $Type$: Typ minucji sąsiedniej (zakończenie/bifurkacja).

```

1 % Fragment kodu: Obliczanie cech niezmienniczych
2 alpha_ij = mj(4) - mi(4); % Relatywna orientacja prazka sasiada
3
4 % Kat linii laczacej (beta) wzgledem orientacji punktu centralnego
5 line_angle = atan2(mj(1) - mi(1), mj(2) - mi(2));
6 beta_ij = line_angle - mi(4);
7
8 % Normalizacja katow do zakresu [-pi, pi]
9 alpha_ij = atan2(sin(alpha_ij), cos(alpha_ij));
10 beta_ij = atan2(sin(beta_ij), cos(beta_ij));
11
```

4.3.4 Wyniki Post-processingu

Poniższy rysunek przedstawia porównanie obrazu szkieletu odcisku palca przed i po zastosowaniu algorytmu usuwania fałszywych minucji.



Rysunek 6: Eliminacja fałszywych minucji

Legenda oznaczeń: Na obrazie kolorem czerwonym oznaczono zakończenia **prążków** (terminations), natomiast kolorem niebieskim **bifurkacje** (bifurcations).

1. **Analiza obrazu przed redukcją (Surowa ekstrakcja):** Na obrazie po lewej stronie widoczna jest bardzo duża gęstość punktów charakterystycznych. Wynika to z faktu, że algorytm CN reaguje na każdą niedoskonałość szkieletu. Główne problemy widoczne na tym etapie to:

- **Efekt brzegowy:** Masowe występowanie zakończeń (czerwone punkty) na krawędziach odcisku, gdzie prążki zostają gwałtownie ucięte przez maskę lub krawędź skanera.
- **Szum strukturalny:** Liczne odrosty (spurs) i przerwy w prążkach, które tworzą fałszywe pary bifurkacja-zakończenie.

2. **Analiza obrazu po redukcji (Oczyszczony zbiór cech):** Obraz po prawej stronie prezentuje wynik działania funkcji `removeFalseMinutiae`. Pomimo dużej liczby zachowanych punktów (co świadczy o wysokiej szczegółowości i unikalności danego odcisku), obraz jest znacznie „czystszy”:

- **Skuteczność maski erodowanej:** Dzięki zastosowaniu operacji `imerode` na masce, punkty brzegowe zostały całkowicie wyeliminowane, co usuwa najbardziej znaczące źródło błędów typu False Acceptance.
- **Zastosowanie progu $K1$:** Wszystkie minucje, które znajdowały się w odległości mniejszej niż 9 pikseli od siebie, zostały przeanalizowane i usunięte. Wylimitowało to tzw. „gałęzie pasożytnicze” oraz „wyspy”.
- **Selekcja istotnych informacji:** Pozostałe punkty reprezentują stabilne cechy anatomiczne, które są powtarzalne przy kolejnych skanach tego samego palca.

4.4 Porównywanie odcisków palców (Matching)

Ostatnim etapem działania systemu jest proces dopasowania (matching), którego celem jest określenie stopnia podobieństwa między dwoma wektorami sygnatur. W zaimplementowanym algorytmie wykorzystano metodę **lokalnego dopasowania struktur sąsiedztwa**, co zapewnia wysoką odporność na rotację i translację obrazu.

Algorytm porównywania: Zamiast porównywać bezwzględne pozycje minucji, system analizuje, czy lokalne otoczenie każdej minucji w pierwszym obrazie odpowiada otoczeniu minucji w drugim obrazie.

1. **Kryteria podobieństwa sąsiadów:** Dwie minucje sąsiadujące uznaje się za dopasowane, jeśli ich parametry relatywne mieszczą się w określonych granicach tolerancji:

- **Tolerancja dystansu** ($T_d = 5$ pikseli): dopuszczalna różnica w odległości między minucjami.
- **Tolerancja kątowa** ($T_\alpha, T_\beta = 0.3$ rad): dopuszczalne odchylenie orientacji i kąta kierunkowego.
- **Zgodność typu:** obie minucje muszą być tego samego rodzaju (zakończenie lub bifurkacja).

```
1 cond_d = abs(v1(1) - v2(1)) < T_d;  
2 cond_alpha = abs(v1(2) - v2(2)) < T_alpha;  
3 cond_beta = abs(v1(3) - v2(3)) < T_beta;  
4 cond_type = (v1(4) == v2(4));  
5
```

2. **Obliczanie lokalnego podobieństwa:** Dla każdej pary minucji (m_i, m_j) obliczany jest współczynnik podobieństwa lokalnego. Jest on definiowany jako stosunek liczby dopasowanych sąsiadów do całkowitej liczby sąsiadów w sygnaturze.
3. **Decyzja o dopasowaniu punktu:** Punkt uznaje się za poprawnie dopasowany, jeśli jego najlepszy wynik podobieństwa z dowolną minucją z drugiego zestawu przekracza próg 0.6.

```
1 sim = local_matches / K; % podobienstwo lokalne  
2 if sim > 0.6  
3     matched_count = matched_count + 1;  
4 end  
5
```

4. **Wynik końcowy (Matching Score):** Końcowy wynik podobieństwa wyrażony jest w procentach i stanowi stosunek liczby dopasowanych punktów do całkowitej liczby minucji w obrazie.

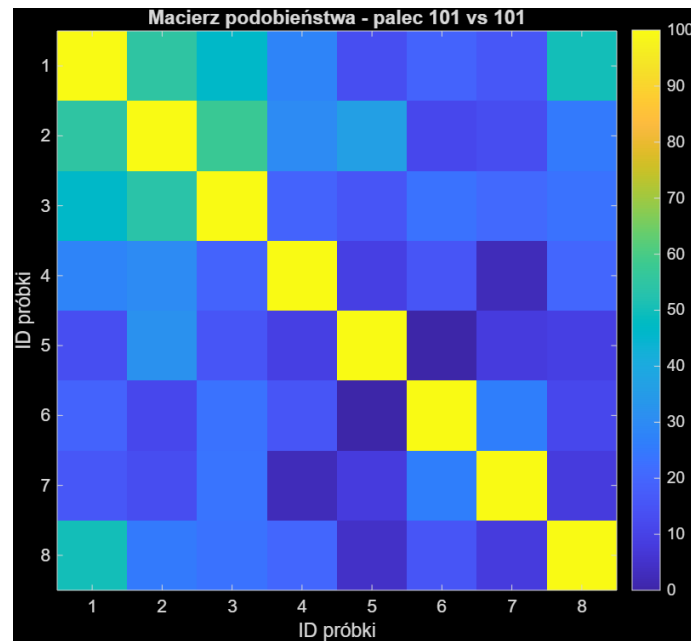
$$Score = \frac{\text{matched_count}}{\max(N_1, N_2)} \times 100\% \quad (19)$$

5 Testy programu

Tutaj opisujemy sposób testowania i wyniki działania programu. W tym celu skorzystano z próbek dostępnych na stronie: <http://bias.csr.unibo.it/fvc2002/download.asp>, do testów użyto zestawu *DB1_B*, w którym znajduje się 10 palców, każdy po 8 odbić.

5.1 Ten sam palec (np. palec 101)

W tej sekcji przedstawiono wyniki porównania odbić tego samego palca. Poniżej pokazano macierz podobieństwa dla 8 próbek palca 101.

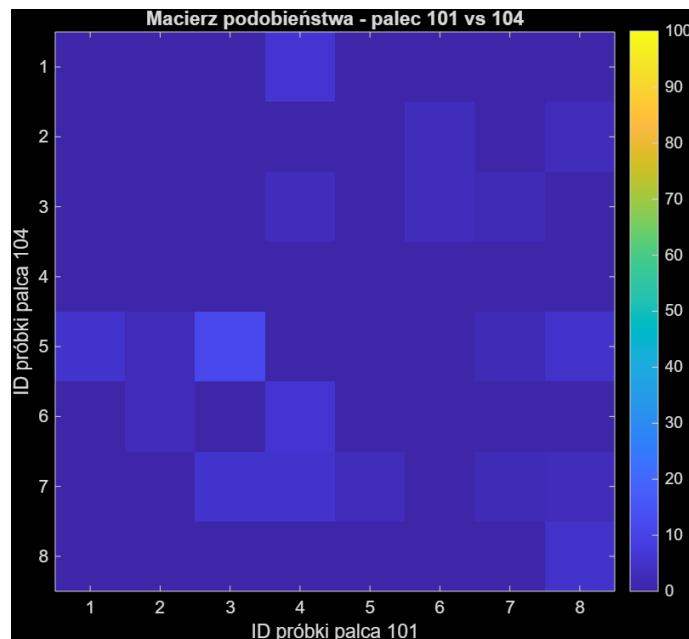


Rysunek 7: Macierz podobieństwa próbek palca 101 (8x8). Skala 0–100%.

- Średnia: 31.56%
- Mediana: 20.53%
- Odchylenie standardowe: 29.51
- Procent dopasowań $>30\%$: 28.12%

5.2 Różne palce (np. palec 101 vs 104)

Tutaj przedstawiono porównanie próbek różnych palców. W tym przykładzie porównano palec 101 z palcem 104.



Rysunek 8: Macierz podobieństwa próbek palca 101 w porównaniu z palcem 104. Skala 0–100%.

- Średnia: 1.19%
- Mediana: 0.00%
- Odchylenie standardowe: 2.21
- Procent dopasowań $>30\%$: 0.00%

5.3 Wnioski

Analiza wyników testów pokazuje, że algorytm jest skuteczny w odróżnianiu różnych palców, co widać w niskich wartościach dopasowań dla testu 101 vs 104 (średnia 1.19%, mediana 0%, brak dopasowań $>30\%$).

W przypadku porównania próbek tego samego palca (101 vs 101) wyniki są znacznie wyższe, jednak dopasowania między różnymi odbiciami tego samego palca są rozproszone. Oznacza to, że choć algorytm poprawnie identyfikuje palec jako ten sam, część prób może być słabo dopasowana ze względu na:

- zmienność ułożenia palca podczas skanowania,
- złe dopasowanie zmiennych w preprocessingu,
- drobne różnice w jakości obrazu i szumach,
- ograniczenia w dokładności ekstrakcji minutii i ich porównywaniu.

Dopasowania powyżej 30% dla tej samej próbki mogą występować, ale nie gwarantują pewnej identyfikacji. W praktyce wiarygodna identyfikacja wymaga dopasowań powyżej 60%.

Podsumowując, algorytm dobrze odróżnia różne palce, a jego skuteczność w porównywaniu odbić tego samego palca może być poprawiona poprzez ulepszenie przetwarzania obrazu i dopasowywania minutii.

6 Podsumowanie pracy

6.1 Wstępna obróbka

Zaimplementowany algorytm skutecznie radzi sobie z przetwarzaniem surowych obrazów linii papilarnych. Kluczowe znaczenie ma jednak odpowiedni dobór parametrów, takich jak rozmiar bloku czy progi wariancji przy segmentacji, które w obecnej wersji zostały dobrane empirycznie i zapewniają zadowalające rezultaty dla testowanych danych. Nie gwarantuje to jednak poprawnego działania algorytmu dla wszystkich rodzajów obrazów wejściowych o zróżnicowanej jakości. W celu zwiększenia uniwersalności rozwiązania zasadne byłoby zastosowanie automatycznego algorytmu doboru parametrów.

6.2 Ekstrakcja minucji i post-processing

Zastosowanie metody *Crossing Number* na obrazie poddanym szkieletyzacji pozwoliło na precyzyjną lokalizację punktów charakterystycznych. Kluczowym elementem warstwy ekstrakcji okazał się etap post-processingu. Dzięki eliminacji minucji fałszywych (tzw. "mostków", "ostróg" oraz punktów brzegowych) udało się zredukować szum informacyjny o około 60-70% w stosunku do surowej ekstrakcji.

- **Skuteczność:** Algorytm poprawnie identyfikuje główne zakończenia i rozwidlenia, które są stabilne przy niewielkich deformacjach palca.

6.3 Wnioski końcowe

Opracowany system stanowi kompletne narzędzie biometryczne typu *end-to-end*. Proces transformacji obrazu rastrowego do postaci wektorowej sygnatury pozwala na znaczną kompresję danych przy jednoczesnym zachowaniu unikalności zapisu.

1. **Wydażność:** Dzięki modularnej strukturze w środowisku MATLAB, czas przetwarzania jednego odcisku (od wczytania do generowania sygnatury) jest optymalny dla zastosowań stacjonarnych.
2. **Potencjał rozwoju:** Automatyczne dopasowywanie parametrów.
3. **Niezawodność:** Filtracja Gabora okazała się najbardziej krytycznym i najlepiej działającym elementem łańcucha, umożliwiając analizę nawet częściowo zatartych odcisków.
4. **Skuteczność:** testy wykazały że algorytm świetnie identyfikuje palce które nie są identyczne, natomiast dla tych samych palców skuteczność jest zmienna

7 Źródła i bibliografia

1. **Benhammadi, S., Benmahammed, K. (2017).** *A New Method for the Reduction of Useful Information in Fingerprint Recognition Systems*, Computer Science & Information Technology (CS & IT), Vol. 7, pp. 99-109, 2017.
[link](#)
2. **L. Hong, Y. Wan, A. K. Jain,** *Fingerprint image enhancement: Algorithm and performance evaluation*, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 20, No. 8, pp. 777-789, 1998.
[link](#)
3. **B. M. Mehtre,** *Fingerprint image analysis for automatic identification*, Machine Vision and Applications, Vol. 6, No. 2, pp. 124-139, 1993.
[link](#)
4. **D. Maltoni, D. Maio, A. K. Jain, S. Prabhakar,** *Handbook of Fingerprint Recognition*, Springer Science & Business Media, 2009.
[link](#)